

Name:Aarya Prasad Lokhande

Python Activity Report

Title

Python Lists

1. Introduction

Python lists are one of the most versatile and commonly used data structures in Python. They allow storage of multiple values in a single variable and support dynamic operations such as insertion, deletion, updating, and traversal. This activity demonstrates practical list operations using a real-world case study of managing student grades.

2. Objective

The objective of this activity is to understand and apply various Python list operations by performing different modifications and analyses on a student grades database.

3. Key Functions and Methods Used

- `append()` – to add new records at the end of the list
- `insert()` – to insert records at a specific index
- `remove()` – to delete a specific record
- `pop()` – to remove the last record
- `count()` – to count duplicate values
- `extend()` – to merge two lists
- `reverse()` – to reverse list order
- `copy()` – to create a shallow copy of the list

- Indexing – to update specific records
 - `clear()` – to remove all records
-

4. Problem Statement

Given a list of student names and their marks, perform various list operations such as adding new students, inserting records at specific positions, removing records, counting duplicate marks, merging another batch of students, reversing the list, creating a backup copy, updating marks, and finally clearing the database.

Initial Dataset: `student_grades = [{"Suresh", 85}, {"Ramesh", 92}, {"Rahul", 78}, {"Mohan", 78}, {"Raj", 60}, {"Arpit", 81}, {"Soham", 80}]`

5. Approach and Methodology

Step 1: Created a list of students with their respective marks using a nested list structure.

Step 2: Appended a new student entry at the end of the list using the `append()` method.

Step 3: Inserted a new student record at a specific index using the `insert()` method.

Step 4: Removed a specific student record from the list using the `remove()` method.

Step 5: Removed the terminal (last) student record using the `pop()` method.

Step 6: Counted duplicate student marks (78) to identify how many students scored the same marks.

Step 7: Merged the existing list with another batch of student records using the `extend()` method.

Step 8: Reversed the list order using the `reverse()` method and created an independent backup using the `copy()` method.

Step 9: Updated the marks of the student at index 2 using direct indexing.

Step 10: Cleared all student records from the database using the `clear()` method.

6. Applications

- Managing student academic records
 - Handling employee or customer databases
 - Processing marks, scores, or survey results
 - Data manipulation in real-world Python applications
-

7. Conclusion

This activity successfully demonstrated the use of Python list operations through a practical student grades management system. Lists provide flexibility and efficiency in handling structured data and are essential for real-world programming tasks.

8. Learning Outcomes

- Understood Python list structure and nested lists
 - Learned to perform insertion, deletion, and update operations
 - Gained experience in data manipulation using built-in list methods
 - Developed problem-solving skills using Python lists
-

End of Report